Specification Games: Witness-Based Specification Strength and the Vacuity Gate as a Reward

Vladimir Stoliarov

October 29, 2026

Abstract

This note studies the vacuity gate of Trusted Code Generation (TCG), which accepts a specification only if it rejects declared faulty variants of the artifact, read as an adversary's move against a specification-writing generator that is paid by a kernel verdict. We argue that the gate has precise mathematical content and that this content yields falsifiable hypotheses about LLM pipelines that write their own specifications. The mutation score, adequacy and the equivalent-mutant problem are standard in mutation analysis; program mutation as a completeness measure for specifications, and the loop in which an LLM refines a specification against a verifier and that score, have been published. What is claimed here is a certified form of the kill-set order on specifications, computed from kernel-checked witnesses; a sequential adversarial evaluation whose payoff is the TCG verdict; four short statements, two about its optima and two about what a certified score guarantees; and hypotheses about what an optimizer does to the metric under pressure. What is not claimed is that LLMs behave as the optimizers in these statements, that any mutant library is complete for a vulnerability class, or that the results transfer beyond executable semantics. This is a research note with an experimental program, not a results paper.

1 Scope

What is borrowed: the mutation score, adequacy and the equivalent-mutant problem are standard in mutation analysis; mutation of programs as a completeness measure for specifications, and the loop in which an LLM refines a specification against a verifier and that score, have been published (Section 2). What is claimed here: a certified form of the kill-set order on specifications, computed from kernel-checked witnesses; a sequential adversarial evaluation whose payoff is the TCG verdict; four short statements, two about its optima and two about what a certified score guarantees; and hypotheses about what an optimizer does to the metric under pressure. What is not claimed: that LLMs behave as the optimizers in these statements (Section 9 measures that), that the mutant library is complete for any vulnerability class (an audited assumption, Section 8), or that the results transfer beyond executable semantics (Section 3).

Notation follows the TCG v2 paper [1] and the v3 paper [2]. Readers who know those can start at Section 4.

2 Related work and what is borrowed

The metric and the feedback loop are prior art; as far as we know, the adversarial reading of them and their placement behind a kernel are not. Table 1 records, for each line of work, what it establishes, what this note takes from it, and what the published material does not describe.

Once the metric and the loop are set aside, four things remain: the payoff structure of Section 7, in which the writer of the specification is paid by the same oracle that measures it; the witness of Section 5 as an object the kernel checks, so the certified score and the certified order of Section 6 are recomputed

Work	What it establishes	What we take	Not described there (as far as the published material shows)
DeMillo, Lipton & Sayward [3]; Jia & Harman [4]	Mutation score, adequacy, equivalent mutants, coupling effect, competent-programr hypothesis	Definitions 5.3–5.4; V-completeness as a strong form of the coupling hypothesis	An optimizing writer; witnesses as certificates
Budd & Gopal [5]; Ammann & Black [6]; Le Traon et al. [7]	Mutation applied to specifications; mutation coverage of specs via model checking; implementation mutants to evaluate Eiffel contracts	The transport from test suites to specifications	Witnesses as certificates; a generator paid by the score
Beer et al. [8, 9]; Kupferman & Vardi [10]; Purandare & Somenzi [11]	Vacuity detection in model checking: replace a subformula by *false* and recheck	The notion that a passing check may be vacuous	A graded measure of strength
IronSpec, Goldweber et al. [12]	Specification mutation testing and spec-testing proofs for Dafny	Evidence that spec weakness is a practical failure mode in verified systems	Program mutants as the probe; an optimizing writer
MutDafny, Amaral, Mendes & Campos [13]	32 program-mutation operators for Dafny; mutants that still verify flag weak specs; 794 real programs, five weak specs found	The program-mutation probe applied to a verifier-checked spec	Kernel-checked witnesses; adversarial or trained writers
Certora Gambit [14]	Mutation testing of formal specifications for EVM contracts, with Prover jobs, dashboards and reports	The domain instance; evidence that the gate is practical on EVM	Witnesses as objects a third party re-checks without the Prover (not described in the public documentation)
SpecGen, Ma et al. [15]	LLM writes JML, OpenJML verifies, errors fed back; mutation of the spec itself to repair it; 385 programs, 72.5% verified	The verifier-in-the-loop setup	Program mutation as a strength measure
FormalBench, Le-Cong, Le & Murray [16]	Mutation analysis as a completeness proxy for LLM-written specs, validated against human labels; notes a slight drop in completeness as verification success rises	The completeness metric; the first hint of H1	Drift as an object of study; incentives
AutoReSpec, Ayon & Ahmed [17]	Verifier-guided JML synthesis with primary and fallback models; mutation-based completeness as the evaluation metric; 67/72 verified, 69.2% completeness	The verifier-feedback loop	Pressure beyond prompting; held-out mutants
AutoJML, Ayon & Ahmed [18]	ReAct agent with OpenJML and MAJOR mutation feedback; 109/120 verified, 79.3% completeness	The feedback loop f_M of Section 9; the aggregate effect H2 is planned to replicate	Per-round dynamics; a comparison against Accept-only feedback; re-verifiable scores
MuTAP, Dakhel et al. [19]	LLM improves tests using mutation-score feedback	Evidence that score feedback moves an LLM	Specifications; verification
PC2, Ciliberto, Rudi & Tassi [20]	Proof-carrying contracts: an agent supplies a machine-checkable proof that each operation preserves agreed constraints, checked by an Airlock	The trust posture: the agent is untrusted and the checker is small	Evidence about the discriminating power of the constraint itself; PC2 proves behavior conforms to a constraint; this note asks whether the constraint says anything

Table 1: Prior work, what is inherited from it, and what it does not describe.

rather than reported; the second layer of Section 8, where the declared mutant library becomes a target and generalization to held-out faults is the measurement; and hypotheses H1, H3, H4 of Section 9 under optimization pressure stronger than a prompt loop. H2, the empirical core of the prior work, is to be replicated here; it is not claimed as new. PC2 and this note are complementary: PC2 certifies that behavior respects a constraint; the gate studied here certifies that the constraint rejects declared faults.

3 Setting

We work inside one verifiable domain $\mathscr{D}$ of v3 [2] (Definition 7), with two additional conditions that make witnesses kernel-checkable. Here "kernel" means the domain's trusted certificate checker: a proof-assistant kernel such as Lean's, or, in an SMT-backed instantiation, the checker that replays a counterexample as a ground execution (Section 10).

From v2/v3 we inherit: artifacts $b \in B$; configurations Σ; a specification $\varphi = (\mathrm{Pre}, \mathrm{Trace}, \mathrm{Post}, \mathrm{Exit})$ over configurations and finite traces; the semantic claim $b \models \varphi$ (every execution of b from a configuration satisfying Pre has a trace satisfying Trace, ends in a configuration satisfying Post, with exit status satisfying Exit); the checker $\mathrm{Check}(b, \varphi, c)$ ($\mathrm{Check}(\varphi, b, a, c)$ of v2, with the annotations a folded into c) with results Accept, $\mathsf{Refute}(w)$, Unknown; and Assumption 3 of v2 [1] (soundness): $\mathrm{Check}(b, \varphi, c) = \mathsf{Accept}$ implies $b \models \varphi$. From v3 we also take the trust classes T (trusted-formal), T^* (trusted non-formal, borne by the human oracle H) and U (untrusted), and the hard, evidence and provenance layers of the bundle with its session ledger (v3, Sections 12, 13 and 17). Observational equivalence $b \equiv b'$ means b and b' produce the same observable traces from every configuration; by construction φ is stated over observables, so $b \equiv b'$ implies $(b \models \varphi \iff b' \models \varphi)$.

Condition E (executable, deterministic, observable semantics)**.** There is a computable function $\mathrm{run} : B \times \Sigma \to \mathit{Traces}$ such that $\mathrm{run}(b, \sigma)$ is the finite, resource-bounded observable trace of b from σ. Determinism is assumed: the environment, the caller, and any scheduler are part of σ, so one configuration fixes one execution; a domain with residual nondeterminism needs a set-valued run and witnesses quantified over that set, which is outside this note. Resource exhaustion (out of gas) is an exit status and is handled by the Exit component of φ, not by exclusion. Traces are observable by construction: $\equiv$ compares $\mathrm{run}(b, \sigma)$ and $\mathrm{run}(b', \sigma)$, so a single configuration on which they differ is a proof of $\not\equiv$. For EVM this is gas-bounded evaluation with the block and call context in σ; in Lean it is a definition the kernel reduces.

Condition D (decidable components)**.** Pre is decidable on configurations; Trace, Post and Exit are decidable on finite traces, final configurations and exit statuses respectively. Then "$\mathrm{run}(b, \sigma)$ violates φ" is a ground proposition the kernel decides by evaluation.

These two conditions are what the gate needs beyond a verifiable domain in the sense of v3 [2] (Definition 7, Section 11). Everything below holds for any domain satisfying them; nothing below uses EVM except that E and D are cheap there.

4 Mutation classes

Sections 4–6 are background: standard mutation analysis transported to specifications, with one change of judge (witnesses checked by the kernel instead of a tool report). A mutation class fixes what the adversary is allowed to do; its V-completeness (Definition 4.3) is the assumption, beyond checker soundness, on which the security content of Section 8 rests.

Definition 4.1 (mutation class)**.** A mutation class at b is a finite, nonempty set M of artifacts $b' \in B$, each identified by $\mathrm{id}(b')$, together with provenance metadata: the operator that produced it and the fault class it is meant to represent. Identity is by hash, so what a witness refers to is fixed without re-applying any operator (Section 10). Two mutants with equal hashes are one element; two with different hashes but equal behavior are counted twice, a bias recorded in Section 11.

Definition 4.2 (properness)**.** M is proper at b if for every $b' \in M$ there is a configuration σ with $\mathrm{run}(b,\sigma) \neq \mathrm{run}(b',\sigma)$. Since traces are observable (Condition E), this is $b' \not\equiv b$. Properness evidence is the pair (b',σ), and the inequality is a ground fact the kernel evaluates.

Properness is where the classical equivalent-mutant problem lives: deciding $b' \equiv b$ is undecidable in general. We do not decide it. We construct M by injecting named faults (remove a balance check, reorder effects and interactions, drop an access control, shift a bound by one), and each injection comes with the σ on which the mutant visibly differs. Two things are then distinct in the bundle: the inequivalence at σ, which the kernel checks and which is therefore T; and the claim that the mutant represents the named fault, which is a label and stays T^*. A mutant that does not compile or deploy is not an artifact and is not in M; it is never counted as killed. An artifact that always reverts is proper and stays in M.

Definition 4.3 (vulnerability class and completeness)**.** A vulnerability class is a set $V \subseteq B$ closed under $\equiv$. M is V-complete on a set $N \subseteq B$ at b if for every $b'' \in N \cap V$ there is $b' \in M$ with $b'' \equiv b'$. N is the neighborhood of interest, for instance the artifacts a generator could produce from the same intent.

V-completeness is a strong assumption: it demands whole-program observational equivalence between every V-vulnerable variant in N and some declared mutant, not membership in a fault family. Two independent reentrancy faults in different functions are not equivalent, so a library with one of them is not complete for reentrancy on any N that contains the other. V-completeness cannot in general be proved inside the framework; it is a signed assumption whose scope is exactly the N it names, and Section 8 states what follows when it holds and nothing when it does not.

5 Witnesses and killing

A witness turns "this mutant is caught" from a claim into a kernel-checked fact; the note keeps two kill sets apart, the semantic one, fixed by the semantics, and the certified one, established by presented witnesses, because only the second is checkable.

Definition 5.1 (witness)**.** A witness against (b,φ) is a configuration σ with $\mathrm{Pre}(\sigma)$ such that $\mathrm{run}(b,\sigma)$ violates φ. Under Conditions E and D the proposition "σ is a witness against (b,φ)" is ground and decidable, so a witness is a certificate the kernel checks by evaluation. In the SMT instantiation of v2 this is the counterexample carried by $\mathsf{Refute}(w)$; in Lean it is a proof term closed by `decide` or `rfl`.

Lemma 5.2 (witness soundness)**.** *If σ is a witness against (b,φ) then $b \not\models \varphi$, hence* $\mathrm{Check}(b,\varphi,c) \neq \mathsf{Accept}$ *for every c.*

Proof. The first part is the definition of $\models$; the second is Assumption 3 of v2. □

Definition 5.3 (semantic and certified kill sets)**.** For a witness set $W \subseteq M \times \Sigma$ presented by an adversary,

$$K^{\mathrm{sem}}_M(\varphi,b) = \{\, b' \in M : \exists \sigma.\ \mathrm{Pre}(\sigma) \wedge \mathrm{run}(b',\sigma) \not\models \varphi \,\},$$
$$K^{\mathrm{cert}}_W(\varphi,b) = \{\, b' \in M : \exists (b',\sigma) \in W.\ \mathrm{Pre}(\sigma) \wedge \mathrm{run}(b',\sigma) \not\models \varphi \,\},$$

with scores $\mathrm{ms}^{\mathrm{sem}} = |K^{\mathrm{sem}}|/|M|$ and $\mathrm{ms}^{\mathrm{cert}} = |K^{\mathrm{cert}}|/|M|$; $|M| > 0$ by Definition 4.1.

Membership in K^{cert} is decided by evaluating each presented pair, so $K^{\mathrm{cert}} \subseteq K^{\mathrm{sem}}$ always (immediate from Definition 5.3). Membership in K^{sem} asks whether a σ exists in Σ, which Conditions E and D do not make decidable; it is established positively by a witness and negatively only by a complete search or a proof. $\mathrm{ms}^{\mathrm{cert}}$ is the completeness measure of FormalBench and AutoJML restated over witnesses; $\mathrm{ms}^{\mathrm{sem}}$ is what that measure estimates.

Definition 5.4 (adequacy, certified adequacy, relative vacuity)**.** φ is M-adequate at b if $K^{\mathrm{sem}} = M$. (φ,c,W) is certified M-adequate at b if $\mathrm{Check}(b,\varphi,c) = \mathsf{Accept}$ and $K^{\mathrm{cert}}_W = M$: the original artifact is accepted and every declared mutant has a checked witness. φ is M-vacuous at b if $K^{\mathrm{sem}} = \emptyset$.

Certified adequacy is the only form the bundle can carry, and it includes Accept on b on purpose: an M-adequate φ with Pre $= \top$ and Post $= \bot$ rejects every mutant and also rejects b, so adequacy alone measures discriminating power, not correctness of b or fidelity to intent.

Two remarks fix the meaning of these definitions.

Killing is semantic; certifying is not. K^{sem} depends on run, M, b and φ only, never on which prover or adversary is used; that is the gate's analogue of capability-invariance (Proposition 1 of v2). But what can be checked for finite M is exactly: positive adequacy (one valid witness per mutant), and any claim of the form "these mutants are certified killed". The claims "this mutant is not killed", "the semantic score is exactly s" and "$\varphi \leq \varphi'$ semantically" are not checkable from witnesses alone. Every experimental quantity in Section 9 is therefore a certified quantity at a stated adversary budget, and is named as such.

M-vacuity is relative to M, and classical vacuity implies it. If Pre is unsatisfiable no σ qualifies, and if Post, Trace and Exit are tautologies no run violates φ; both give an empty kill set. The converse fails: a specification with real content that no mutant in M violates is also M-vacuous. So M-vacuity does not replace the separate checks that Pre is satisfiable (the Pre witness of v3, Sections 14.1 and 16) and that the conclusion is non-trivial; it adds a library-relative check beside them, and the bundle keeps all three.

6 Witness-based strength

Specification strength is ordered by what the kill set contains rather than by logical entailment; the order on certified kill sets is checkable, the order on semantic kill sets is what it approximates.

Definition 6.1 (strength orders)**.** For specifications over the same b and M, $\varphi \leq_M \varphi'$ iff $K^{\text{sem}}(\varphi) \subseteq K^{\text{sem}}(\varphi')$; for witness sets W, W', $\varphi \leq_W \varphi'$ iff $K^{\text{cert}}_W(\varphi) \subseteq K^{\text{cert}}_{W'}(\varphi')$. Strict and equivalent forms are written $<$, $\equiv$ with the same subscripts. Propositions 6.2 and 6.3 below are stated for $\leq_M$; their proofs go through unchanged for $\leq_W$ when the same witnesses are presented on both sides.

The relation $\leq_M$ is a preorder; the map $\varphi \mapsto K^{\text{sem}}_M(\varphi, b)$ sends it into the subset lattice of M. The trivially true specification $\varphi_\top$ (Pre true, every conclusion a tautology) is a minimum: its kill set is empty. Every M-adequate φ is a maximum.

Proposition 6.2 (logical strength refines $\leq_M$)**.** *Write $\varphi' \Rightarrow \varphi$ if every trace satisfying φ' satisfies φ and* $\text{Pre}_{\varphi'} \supseteq \text{Pre}_{\varphi}$. *Then $\varphi' \Rightarrow \varphi$ implies $\varphi \leq_M \varphi'$.*

Proof. A witness against (b', φ) for $b' \in M$ is a σ with $\text{Pre}_\varphi(\sigma)$ and $\text{run}(b', \sigma) \not\models \varphi$. Then $\text{Pre}_{\varphi'}(\sigma)$, and $\text{run}(b', \sigma) \not\models \varphi'$ since otherwise it would satisfy φ. So σ witnesses against (b', φ'). □

The converse fails. Take a token with a supply cap and $M = \{m_1\}$ where m_1 removes the access control on `mint`. Let φ_1 say total supply changes only through `mint` called by the owner, and φ_2 say total supply never exceeds the cap. Neither entails the other, but both kill m_1 (a configuration where a non-owner mints past the cap witnesses both), so $\varphi_1 \equiv_M \varphi_2$. The order $\leq_M$ does not see logical differences that no mutant in M exposes. That is a feature for measurement and a limitation for security; Section 8 makes the limitation precise.

Proposition 6.3 (conjunction)**.** $K^{\text{sem}}_M(\varphi \wedge \psi, b) = K^{\text{sem}}_M(\varphi, b) \cup K^{\text{sem}}_M(\psi, b)$ *when φ and ψ share* Pre.

Proof. A run violates $\varphi \wedge \psi$ iff it violates φ or violates ψ. □

Consequently adding a property never lowers strength, and removing a conjunct lowers it exactly when that conjunct was the only one killing some mutant. "Does this property carry weight" becomes "is $K(\varphi)$ strictly larger than $K(\varphi$ without it)". For certified sets this is a set difference the kernel confirms witness by witness; for semantic sets it also needs a proof that the mutant is not killed without the conjunct, which witnesses alone cannot give (Section 5).

7 The specification game

The "game" is a sequential adversarial evaluation with a stated payoff; no equilibrium result is claimed, and the word is kept only because the payoff is what the generator is paid.

Players. The generator G (untrusted) chooses the specification. The adversary A (untrusted) chooses configurations for the declared mutants, that is, pairs in $M \times \Sigma$; M itself is declared by the producer. The kernel (trusted) evaluates. In the spec-only game studied here the artifact b and the natural-language intent I are fixed; G's job is to write φ for b. The two-sided game where G also chooses b is noted in Section 11.

Moves. G moves first with a pair (φ, c). A moves second with a finite witness set $W \subseteq M \times \Sigma$, produced by a search procedure with a stated budget (fuzzing, symbolic execution, a model); A has no objective of its own in this note. The kernel then evaluates. A's search may be incomplete; Theorem 8.4 states the consequence for the score.

Payoffs and two assumptions. Let $\mathrm{acc}(\varphi, c) = 1$ if $\mathrm{Check}(b, \varphi, c) = \mathsf{Accept}$, else 0. Let $\kappa(\varphi, c) \geq 0$ be a cost.

Assumption TA (trivial acceptance)**.** There is $c_\top$ with $\mathrm{Check}(b, \varphi_\top, c_\top) = \mathsf{Accept}$.

Assumption CS (cost separates content)**.** $\kappa(\varphi_\top, c_\top) \leq \kappa(\varphi, c)$ for all (φ, c), and strictly whenever $K^{\mathrm{sem}}(\varphi) \neq \emptyset$.

Neither follows from soundness. For the pilot, κ is the number of non-tautological conjuncts in φ, which makes Assumption CS hold; Assumption TA is a property of the checker and is assumed. Three idealized payoffs differ only in what G is paid, all computed on the semantic kill set:

$$\begin{aligned} \Gamma_\varnothing: &\quad u_G(\varphi, c) = \mathrm{acc}(\varphi, c) - \kappa(\varphi, c), \\ \Gamma_M^{\mathrm{hard}}: &\quad u_G(\varphi, c) = \mathrm{acc}(\varphi, c) \cdot [K_M^{\mathrm{sem}}(\varphi, b) = M] - \kappa(\varphi, c), \\ \Gamma_M^{\mathrm{soft}}: &\quad u_G(\varphi, c) = \mathrm{acc}(\varphi, c) \cdot \mathrm{ms}_M^{\mathrm{sem}}(\varphi, b) - \kappa(\varphi, c). \end{aligned}$$

$\Gamma_\varnothing$ rewards Accept alone; Γ_M^{hard} rewards Accept only when every mutant is killed; Γ_M^{soft} pays by score. These are the objectives the theorems of Section 8 describe. What is actually paid is the implemented objective $\hat{u}$, obtained by replacing K^{sem} with K_W^{cert} for the W that A presented; $\hat{u} \leq u$ in the gated games by Theorem 8.4, and $\hat{u} = u$ in $\Gamma_\varnothing$. Theorems 8.1–8.3 are about u; Theorem 8.4 is the bridge; the experiments observe $\hat{u}$.

Parameters that are not part of the mathematics but decide the experiment. The feedback function f that G observes between rounds (Section 9); the number of rounds R and the best-of-k selection that stand in for an optimizer; the cost model κ; and the wording of the intent I, which must state that the game is about discriminating power, not about making the checker pass. These parameters, with the wording of I, answer the objection that a reasonable generator simply repairs φ correctly: the experiment varies f, not the generator's goodwill, and the theorems describe what each f rewards.

8 Theorems

The four theorems are short. The first two describe incentives; the third carries the security content, and its part (ii) is conditional on V-completeness; the fourth bounds what an incomplete adversary can do.

Theorem 8.1 (degenerate optimum of Accept-only reward)**.** *Under Assumptions TA and CS, $(\varphi_\top, c_\top)$ is an optimal strategy for G in $\Gamma_\varnothing$, and every optimal strategy has an empty semantic kill set.*

Proof. By Assumption TA, $\mathrm{acc}(\varphi_\top, c_\top) = 1$, so $u = 1 - \kappa(\varphi_\top, c_\top)$, and by Assumption CS no strategy exceeds this. Any strategy with nonempty kill set has strictly larger κ by Assumption CS and so is not optimal. □

What the theorem says: a reward that observes only Accept has a vacuous optimum. What it does not say: that a given generator will find that optimum. That is Hypothesis H1 in Section 9.

Theorem 8.2 (what the gate makes necessary)**.** *(a) In Γ_M^{hard}, every strategy with positive payoff is Accepting and M-adequate. If some Accepting M-adequate strategy has $\kappa < 1$ and the maximum is attained, every maximizer is M-adequate.*

(b) In Γ_M^{soft}, every Accepting maximizer is Pareto-optimal in $(|K^{\mathrm{sem}}|, -\kappa)$ among Accepting strategies. If, in addition, Assumptions TA and CS hold, some Accepting strategy is M-adequate, and the marginal cost of each further kill is below $1/|M|$ (for Accepting $\varphi <_M \varphi'$, $\kappa(\varphi') - \kappa(\varphi) < (|K^{\mathrm{sem}}(\varphi')| - |K^{\mathrm{sem}}(\varphi)|)/|M|$), then every maximizer is Accepting and M-adequate.

(c) If $M_1 \subseteq M_2$, every M_2-adequate specification is M_1-adequate.

Proof. (a) Positive payoff needs acc $= 1$ and the indicator equal to 1, which is adequacy. A non-adequate strategy has payoff $-\kappa \le 0$; the named adequate strategy has $1 - \kappa > 0$; so no non-adequate strategy is a maximizer. (b) If an Accepting maximizer were Pareto-dominated among Accepting strategies, the dominating strategy would have weakly larger score and weakly smaller cost with one strict, hence strictly larger payoff. For the second claim, let φ' be the Accepting M-adequate strategy. Since $|M| > 0$, $\varphi_\top <_M \varphi'$, and $\varphi_\top$ is Accepting by Assumption TA, so the marginal-cost condition gives $\kappa(\varphi') - \kappa(\varphi_\top) < 1$. Hence $u(\varphi') = 1 - \kappa(\varphi') > -\kappa(\varphi_\top) \ge -\kappa(\varphi)$ for every φ by Assumption CS, and a non-Accepting strategy, whose payoff is $-\kappa$, is not a maximizer. For a non-adequate Accepting φ, $K^{\mathrm{sem}}(\varphi) \subsetneq M = K^{\mathrm{sem}}(\varphi')$, so $\varphi <_M \varphi'$ and the condition gives $u(\varphi') - u(\varphi) = (|M| - |K^{\mathrm{sem}}(\varphi)|)/|M| - (\kappa(\varphi') - \kappa(\varphi)) > 0$; so φ is not a maximizer either. (c) Inclusion of kill sets. □

Without these conditions maximizers need not be adequate: a hard game whose only adequate strategy costs 1.2 loses to the trivial spec at payoff 0, and in a soft game a cheap one-kill spec at payoff 0.4 beats a full-kill spec at 0.2. The stated conditions exclude both. One corner remains: under a steep κ the soft optimum leaves expensive mutants alive. That is incompleteness, not gaming, and the experiment separates the two by reporting kill sets, not scores.

Theorem 8.3 (what certified adequacy guarantees)**.** *Let (φ, c, W) be certified M-adequate at b.*

(i) Unconditionally: $b \models \varphi$, and every $b' \in M$ has $b' \not\models \varphi$, so no certificate can make Check *accept (b', φ).*

(ii) If moreover M is V-complete on N at b, then every $b'' \in N \cap V$ has $b'' \not\models \varphi$, and $\mathrm{Check}(b'', \varphi, c') \neq$ Accept *for every c'.*

Proof. (i) Accept gives $b \models \varphi$ by Assumption 3 of v2; each $b' \in M$ has a checked witness, so $b' \not\models \varphi$ by Lemma 5.2. (ii) Take $b'' \in N \cap V$; V-completeness gives $b' \in M$ with $b'' \equiv b'$; φ is stated over observables, so $b'' \not\models \varphi$; Assumption 3 of v2 gives the last claim. □

Part (i) is what the bundle proves and is modest: the accepted specification separates b from the declared mutants. Part (ii) is where any security reading lives, and it is only as strong as the V-completeness claim, which is why Section 10 records that claim as a signed T^* object with its N. The word "checkable" is reserved for the case where N is finite, enumerated in the bundle, and each element's equivalence to a declared mutant is itself proved and the proof kernel-checked; otherwise V-completeness is an assumption and the theorem is conditional.

Theorem 8.4 (one-sided adversary error)**.** *Let A present witness set W, and let $K_W^{\mathrm{cert}}(\varphi, b)$ be the mutants for which W contains a valid witness. Then $K_W^{\mathrm{cert}}(\varphi, b) \subseteq K_M^{\mathrm{sem}}(\varphi, b)$: the certified score is a lower bound on the semantic score.*

Proof. Every presented pair in K^{cert}_W satisfies the condition defining K^{sem}_M (Definition 5.3). □

Two readings follow. First, over a fixed, declared M with a kernel-checked numerator, an incomplete adversary cannot inflate the score: a certified score of 1 means a valid witness for every declared mutant, whoever found them. What an incomplete adversary produces is a low score that is ambiguous between a weak specification and a weak search; the bundle resolves the ambiguity only by recording the search budget. Second, the thing that can be gamed is therefore not the score over M but the choice of M: a generator that sees M, or is trained on scores over M, can learn to kill those artifacts rather than the faults they stand for. That is measured not by comparing a reported score to a true score over the same set, which is unavailable because the semantic score cannot be checked (Section 5), but by comparing scores over disjoint sets: s_{public} over the declared M and s_{heldout} over a held-out library M_{heldout} the generator never saw. This is the second layer of the game, and H3–H4 are its measurements.

9 From theorems to hypotheses

The theorems say what each reward makes optimal; the hypotheses say whether an LLM under iterative feedback moves toward that optimum. The *pilot* is the first experiment: three contracts (a capped token, a vault with withdrawal, an escrow), each with a hand-written library of fault-injected mutants, in the prompt regime only; it tests H1 and H2. H3 and H4 need the training regime and form a second stage.

Feedback functions. Between rounds G observes $f(\varphi_t, c_t)$. Three conditions:

- $f_\varnothing$: the Accept bit only (plus compiler or checker errors).
- f_M: the Accept bit and the certified score $\text{ms}^{\text{cert}}_W$, without naming which mutants survived.
- f_{M+}: as f_M, plus the identity of surviving mutants.

Optimization pressure comes in two regimes; H3 and H4 need the second. The prompt regime is R rounds of revision with best-of-k selection; this is the AutoJML setting, where the generator is cooperative and the score is a measurement. The training regime is rejection sampling or fine-tuning on the same payoff; here the score is a reward, and H3 predicts that the generator learns to exploit what the reward does not see.

Weakening event. All per-round quantities are certified, computed at a fixed adversary budget B_A applied identically to every round and condition. A round t is a weakening if $K^{\text{cert}}(\varphi_{t+1}) \subsetneq K^{\text{cert}}(\varphi_t)$; a strengthening if the reverse; a shift if the sets are incomparable. Shifts are reported separately and not folded into either; $|K^{\text{cert}}|$ is reported alongside as the scalar view. To keep adversary variance from counting as weakening, every round is scored against the common witness pool $W^* = \bigcup_t W_t$: each presented pair is re-evaluated against each φ_t, which is sound because a pair (b', σ) does not depend on φ. Events are decided from witnesses, not by reading φ, which removes the judgment call in "the model rewrote the property".

Metrics. Per trajectory: the score $\text{ms}^{\text{cert}}_W(\varphi_t)$ over t; the number of weakening events; whether the final φ has an empty certified kill set; the final kill set as a set. Per condition: the distribution of these over seeds and contracts.

Hypothesis (H1: vacuity drift)**.** Under $f_\varnothing$ the expected score is non-increasing in t, and weakening events concentrate in rounds following a non-Accept. Motivation from Theorem 8.1: the cheapest repair to a specification that fails on b is to weaken it. The FormalBench study [16] records a slight drop in completeness as verification success rises; H1 asks whether that drop is a trajectory with a mechanism, measured by kill sets rather than by an aggregate. Falsification criterion: no difference in weakening rate between $f_\varnothing$ and f_M at the pilot's sample size, in either regime.

Hypothesis (H2: gate reversal; a replication)**.** Under f_M the expected score is non-decreasing in t and the final rate of empty certified kill sets is lower than under $f_\varnothing$. AutoJML [18] reports the aggregate effect in

the prompt regime for Java/JML; per-round dynamics and a comparison with $f_\varnothing$ are not described there (Table 1). We plan to replicate it on EVM artifacts to calibrate the harness and to check that the result survives a kernel-checked score; a failure to replicate would be the finding.

Hypothesis (H3: overfitting to the declared library)**.** Setup: a fault taxonomy is fixed first; two libraries are then built from it independently, M_{public} and M_{heldout}, by separate procedures (a second person, or an operator set fixed and hashed before training starts) through the same materialization pipeline, with properness evidence for every element; M_{heldout} is never shown to G, never used for selection, and scored with the same definition and the same B_A. Prediction, training regime: the gap $s_{\text{public}} - s_{\text{heldout}}$ grows with training steps under f_M, and faster under f_{M+}. The same gap under $f_\varnothing$ is the control. This is the corner of Theorem 8.2 under pressure, and it is a question the work in Table 1 does not ask, because none of it trains on the score.

Hypothesis (H4: false acceptance on held-out faults)**.** The security-facing form of H3: among held-out mutants, the fraction with no certified witness against the final φ at budget B_A (a proxy for false acceptance of a defective variant) is higher for generators trained under f_M than for those under $f_\varnothing$ at matched s_{public}. Three scores are kept apart throughout: s_{public} over M_{public}, s_{heldout} over M_{heldout}, and s_{full} over their union; none is a "true" score, and Theorem 8.4 is a statement about each one separately, never a comparison between them.

Minimum protocol. Fixed before any run: the reward and κ (number of non-tautological conjuncts), R and k for the prompt regime, the training recipe and checkpoint-selection rule for the training regime of the second stage (selection on s_{public} only), seeds, models, B_A, the three contracts and both libraries with hashes. Primary metric: s_{heldout} at the final round or checkpoint; secondary: weakening rate, final rate of empty certified kill sets, s_{public}. Comparisons are paired across seed $\times$ contract with bootstrap intervals; the pilot's sample size is stated as a limit. Policies: a mutant that does not compile or deploy is excluded from the library before the experiment; Check returning Unknown counts as non-Accept; a witness that fails the kernel is discarded and logged, never counted.

What the pilot cannot establish. Generality beyond the three contracts; behavior under fine-tuning (H3, H4); transfer to a Lean oracle.

10 Placement in TCG

The certified score is a re-verifiable quantity of the bundle rather than a claim about it, and the verifier contract below is enough to recompute it without TCG v2/v3 in hand. Table 2 assigns each object the game uses a trust class.

Verifier contract. Inputs: manifest; b; c; φ; semantics version; M as a list of (hash, bytes); properness pairs; W; signatures. Checks, in order:

(1) every hash in the manifest matches its bytes, including the semantics and the checker's own source;
(2) $\text{Check}(b, \varphi, c) = \text{Accept}$;
(3) for each $b' \in M$, some properness pair evaluates to inequality;
(4) for each $(b', \sigma) \in W$, $\text{Pre}(\sigma)$ holds and $\text{run}(b', \sigma)$ violates φ, evaluated under the same semantics as (2);
(5) $\text{ms}_W^{\text{cert}} = |\{b' : \text{some pair passed (4)}\}|/|M|$;
(6) the V-completeness claim, if present, is signed and names its N.

Output: the certified score, the list of mutants without a valid witness, and the list of discarded pairs. Nothing in (1)–(6) applies a mutation operator or consults the adversary: identity is a hash and the operator is a label.

Object	Trust class	Where	What the verifier does with it
Original artifact b, certificate c	T after kernel check	Hard layer	Runs Check(b, φ, c); Accept required for certified adequacy
Mutant artifacts $b' \in M$, by hash	T for identity after hash check	Evidence layer	Recomputes id(b') against the manifest; never applies an operator
Fault labels and operator provenance of each mutant	T^*	Evidence layer	Read as metadata; affects no verdict
Properness evidence (b', σ)	U until checked; then a checked fact	Evidence layer	Evaluates run$(b, \sigma) \neq$ run(b', σ)
Witnesses $W = \{(b', \sigma)\}$	U until checked; then a checked fact, for the statement "Pre(σ) and run(b', σ) violates φ" only	Evidence layer	Evaluates each; discards and logs failures
Certified score $\mathrm{ms}_W^{\mathrm{cert}}$	Derived	Manifest, recomputable	Recomputes from checked W; a stated score that disagrees is rejected
Adversary identity and budget B_A	U	Provenance layer	Read; needed to interpret a low score
V-completeness claim with its N	T^*, signed by H	Evidence layer	Read as an assumption; absent $\to$ named in `not_claimed`
Feedback function and rounds	U	Session ledger	Read for context

Table 2: Trust classes of the objects the game places in the bundle.

In Lean the witness is a closed proof term: given constants `b'` (whose Keccak-256 hash the checker matches to the manifest) and σ, the term has type `Pre` $\sigma \wedge \neg$ `Sat` φ `(run b' `σ`)`, where `Sat` decides Trace, Post and Exit on a finite trace and `run` is the executable semantics; both conjuncts close by `decide`, and `#print axioms` on the term must list only the standard axioms. In the SMT instantiation the counterexample of Refute is replayed by the checker as a ground execution under (4), so the solver's report is never trusted, only re-executed.

11 Open questions

These are the places where the note is most likely wrong or thin; comments on them are the most useful.

1. **Is $\leq_M$ the right coarsening?** It treats every mutant as equal. A weighted order by vulnerability class, or a partial order over a family of classes M_V, would separate "strong against reentrancy, weak against access control". Does weighting buy anything the kill set as a set does not already show?

2. **Can V-completeness be made checkable for concrete V?** For reentrancy expressed as a Trace predicate, the set of fault injections that produce a reentrant call may be finite and enumerable for a given b. If so, Theorem 8.3(ii) becomes unconditional for that V on that neighborhood. For which V does this work?

3. **What is κ for an LLM?** The theorems assume Assumption CS: the trivial specification is cheapest, strictly so against any specification with a non-empty kill set. Candidates: number of properties, length of φ, number of rounds to Accept. The experiment can test which, if any, the generator responds to.

4. **Does the soft game have a hidden degenerate optimum?** Theorem 8.2 notes the corner where expensive mutants are left alive. Is there a cost regime in which the soft optimum is strictly below adequacy for every proper M, and does f_{M+} make it worse (H3)?

5. **The two-sided game.** When G also chooses b, M must be defined relative to the chosen b, and G can choose a b for which M is accidentally improper. Is there a formulation where M is a function of the intent I rather than of b?

6. **Who audits the adversary?** Theorem 8.4 moves gaming to the choice of M. A second gate on the adversary (held-out mutants, as in H4) is one answer; is there a fixed point, or does every gate add a layer?

7. **Unkillable proper mutants.** Properness is checked on full observable traces, but the specification language may not express every observable (gas used, for example). A proper mutant that differs from b only in such an observable is killed by no φ, so the score has a ceiling below 1 and adequacy is unattainable. Should the bundle carry a ceiling estimate, and how would it be computed without deciding which differences the language can express?

8. **Transport to non-executable semantics.** Without Condition E, witnesses are not ground and the gate degrades to "the weakened spec is provable on the mutant", which shows weakness but not strength. Is there a weaker notion of strength that survives?

9. **What is left once AutoJML is subtracted?** The feedback loop and the completeness metric are theirs. Is the game plus the training regime plus kernel-checked witnesses enough for a contribution, or does the note need one more theorem with content (a checkable V-completeness for a concrete V, question 2) to stand on its own?

10. **Duplicate and nondeterministic cases.** Definition 4.1 counts behaviorally equal mutants with different hashes twice, which inflates $|M|$ and gives duplicated behaviors double weight in every score. Is a hash-of-observable-trace-on-σ a usable dedup key, and what does the witness become in a domain where run is set-valued?

11. **Submission type.** With H1–H4 unmeasured, this is a research note with an experimental program. A working prototype of generation $\to$ mutant materialization $\to$ kernel witness check $\to$ recomputable score is needed before the note can be called a research paper; until that exists it is submitted, if at all, as a workshop or agenda paper.

12 Claim ledger

Table 3 lists each claim in this note with its status, so a reader can see at a glance what is proved, what is assumed and what is to be measured.

In Table 3, the first four rows are inherited; the next eleven are the mathematics, of which Theorem 8.3(ii) is conditional on V-completeness; the next four are what a bundle carries and in which trust class; the last five are hypotheses: the pilot measures H1 and H2 (H2 a replication), H3 and H4 belong to the second stage, and transfer to a Lean oracle is untested.

References

[1] V. Stoliarov. Trusted Code Generation: A formal trust-boundary framework for specification-conditioned smart-contract synthesis and verification. Manuscript, version 2, 27 June 2026.

[2] V. Stoliarov. Trusted Code Generation: A formal trust-boundary framework for specification-conditioned smart-contract synthesis and verification. Version 3: Proof-carrying bundles for critical infrastructure. Manuscript, 29 October 2026.

[3] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programr. *IEEE Computer*, 11(4):34–41, 1978.

[4] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[5] T. A. Budd and A. S. Gopal. Program testing by specification mutation. *Computer Languages*, 10(1):63–73, 1985.

Claim	Status	Rests on
Mutation score, adequacy, equivalent mutants	Standard [4]	Section 2
Program mutation as a strength measure for specifications	Standard [5, 6, 12, 13, 16]	Section 2
LLM refines a spec against verifier feedback	Published [15, 17]	Section 2
LLM refines a spec with mutation-score feedback	Published [18]	Section 2
Presented witnesses are kernel-checkable ground facts	Lemma 5.2	Conditions E and D, Assumption 3 of v2
Semantic kill set is prover- and adversary-independent; only the certified kill set is checkable	Definition 5.3 and the remark after it	Condition E
Mutation-relative vacuity is implied by, not equivalent to, classical vacuity	Observation (Section 5)	Definition 5.3
Logical strength refines $\leq_M$; the converse fails	Proposition 6.2 with counterexample	Definition 6.1
Kill sets of conjunctions are unions	Proposition 6.3	Definition 5.3
Accept-only reward has a vacuous optimum	Theorem 8.1	Assumptions TA and CS
Positive payoff in the hard gate requires adequacy; maximizers are adequate under a cost bound	Theorem 8.2(a)	$\kappa < 1$ for some adequate strategy
Accepting soft-gate maximizers are Pareto-optimal; all maximizers are Accepting and adequate under a marginal-cost bound	Theorem 8.2(b)	Assumptions TA and CS; an Accepting adequate strategy; marginal cost per kill below $1/\lvert M\rvert$
Certified adequacy accepts b and rejects every declared mutant	Theorem 8.3(i), unconditional	Definition 5.4
Certified adequacy rejects every V-vulnerable variant in N	Theorem 8.3(ii), conditional	V-completeness of M on N (signed T^*)
Certified score is a lower bound on semantic score; cannot be inflated by a weak adversary	Theorem 8.4	Lemma 5.2
Mutants are proper	Kernel-checked per mutant	Properness pairs
Mutant identity	Hash check	Manifest
Fault labels represent the named faults	Assumption, T^*	Producer
M is V-complete on N	Assumption, signed T^*	Human oracle H
Generators drift to vacuity under Accept-only feedback	Hypothesis H1 (hinted by [16])	Pilot (prompt regime); second stage (training regime)
Score feedback raises completeness in the prompt regime	Hypothesis H2 (replication of [18])	Pilot, prompt regime
Training on the public score widens the public/held-out gap	Hypothesis H3	Second stage, training regime
False acceptance on held-out faults rises with score training	Hypothesis H4	Second stage, training regime
Results transfer to a Lean oracle	Hypothesis, untested	Conditions E and D in Lean

Table 3: Claim ledger.

[6] P. Ammann and P. E. Black. A specification-based coverage metric to evaluate test sets. In *Proc. 4th IEEE International Symposium on High-Assurance Systems Engineering (HASE)*, 1999.

[7] Y. Le Traon, B. Baudry, and J.-M. Jézéquel. Design by contract to improve software vigilance. *IEEE Transactions on Software Engineering*, 32(8):571–586, 2006.

[8] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient detection of vacuity in ACTL formulas. In *Computer Aided Verification (CAV)*, LNCS 1254, 1997.

[9] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient detection of vacuity in temporal model checking. *Formal Methods in System Design*, 18(2):141–163, 2001.

[10] O. Kupferman and M. Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer*, 4(2):224–233, 2003.

[11] M. Purandare and F. Somenzi. Vacuum cleaning CTL formulae. In *Computer Aided Verification (CAV)*, LNCS 2404, 2002.

[12] E. Goldweber, W. Yu, S. A. Vakil Ghahani, and M. Kapritsos. IronSpec: Increasing the reliability of formal specifications. In *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[13] I. Amaral, A. Mendes, and J. Campos. MutDafny: A mutation-based approach to assess Dafny specifications. In *Proc. 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2511.15403.

[14] Certora. Gambit: Mutation testing for Solidity, and *Using Gambit with the Prover*. `https://docs.certora.com/en/latest/docs/gambit/mutation-verifier.html`, 2023.

[15] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu. SpecGen: Automated generation of formal program specifications via large language models. arXiv:2401.08807, 2024.

[16] T. Le-Cong, B. Le, and T. Murray. Can LLMs reason about program semantics? A comprehensive evaluation of LLMs on formal specification inference. In *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[17] R. S. Ayon and S. Ahmed. AutoReSpec: A framework for generating specification using large language models. In *Proc. ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2026. arXiv:2604.03758.

[18] R. S. Ayon and S. Ahmed. AutoJML: Generation and verification of JML specifications using LLM agents. IEEE, 2026. DOI: 10.1109/MPULS.2026.3659250.

[19] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais. Effective test generation using pre-trained large language models and mutation testing. arXiv:2308.16557, 2023.

[20] C. Ciliberto, A. Rudi, and E. Tassi. Proof-carrying contracts: Commitments AI agents cannot break. arXiv:2610.28484, 2026.